

Contents

Waveshare Support Report — 10.85inch e-Paper HAT+ (G)	1
Summary	1
Hardware tested	1
Software stacks tried — all fail identically	1
Observed behaviour by driver	2
Electrical measurements	2
Reset timing (per Waveshare FAQ) — tested, no effect	3
What has been ruled out	3
Questions for Waveshare	3
Environment	4

Waveshare Support Report — 10.85inch e-Paper HAT+ (G)

Product: 10.85inch e-Paper HAT+ (G), 1360×480, 4-colour (black/white/red/yellow) **Symptom:** Panel never displays anything. No flash, no flicker, no partial change, under any condition. **Status:** Reproduced after full RMA replacement of both panel and driver HAT.

Summary

The panel accepts SPI commands but never completes power-on. Specifically:

- After `Reset()`, BUSY reads idle.
- On command `0x04 (POWER_ON)`, BUSY asserts busy.
- **BUSY never releases.** The wait loop runs indefinitely.

This has been reproduced across **2 panels, 2 HAT+ driver boards, 3 Raspberry Pi models, 2 OS installations, and 5 independent software stacks** — including Waveshare’s own C and Python demos, and a demo supplied directly by Waveshare support.

Electrical measurement confirms BUSY (GPIO24) is **actively driven low**, not floating.

Hardware tested

Component	Units tried
10.85" (G) panel	2 — original + RMA replacement
10.85" e-Paper HAT+ (G)	2 — original + RMA replacement (shipped together)
Raspberry Pi	3 — Zero 2 W (BCM2710A1), Zero W (BCM2835), and 3 Model B Rev 1.2 (BCM2837)
Power supply	2 — dedicated 5V/2.5A adapter, and powered USB hub
Cabling	Both the factory interposer + ribbon assembly, and panel FPC connected directly to the HAT (as suggested by Waveshare support)

Interface Config switch confirmed at position **0 (4-line SPI)** throughout.

Software stacks tried — all fail identically

1. **Waveshare Python** `epd10in85g.py` + `DEV_Config*.so` (bit-banged SPI via ctypes)

2. **Waveshare C demo** — E-paper_Separate_Program/10.85inch_e-Paper_G/RaspberryPi/c, built with WiringPi 3.18
3. **Waveshare support-supplied demo** — RPI.zip, EPD_10in85 mono driver, WiringPi backend
4. **Raw Python** — spidev + RPi.GPIO, replicating the documented init sequence manually
5. **Third-party pure-Python driver** — hardware SPI on /dev/spidev0.0 and /dev/spidev0.1 with kernel chip-select, gpiozero for GPIO, no compiled library at all

Stacks 1–4 use manual CS toggling and bit-banged SPI through the precompiled .so. Stack 5 uses an entirely different architecture (kernel hardware SPI, hardware CS). All five produce the same result.

Observed behaviour by driver

(G) driver (epd10in85g.py) — ReadBusyH() waits while BUSY == 0:

```
EPD init...
bcm2835 init success !!!
e-Paper busy H
e-Paper busy release      <- first wait, after Reset(), returns immediately
e-Paper busy H           <- second wait, after command 0x04 (POWER_ON)
[hangs indefinitely]
```

Mono driver (EPD_10in85, from support) — ReadBusy() waits for BUSY == 1:

```
Debug: EPD_10in85_test Demo
set wiringPi lib success !!!
Debug: e-Paper Init and Clear...
Debug: e-Paper busy
[hangs indefinitely]
```

Both are consistent with BUSY being held in the busy state permanently after power-on is commanded.

Electrical measurements

HAT ID EEPROM reads correctly — the Pi communicates with the board:

```
$ cat /proc/device-tree/hat/product /proc/device-tree/hat/vendor
Waveshare 10.85inch e-Paper HAT+ (G)
WAVESHARE
```

SPI enabled, both devices present:

```
$ ls /dev/spidev*
/dev/spidev0.0 /dev/spidev0.1
```

BUSY (GPIO24) is actively driven low. Read directly from SoC registers with pinctrl, no library in between:

```
$ pinctrl get 24
24: ip    -- | lo // GPIO24 = input
```

Commanding an internal pull-up does not change it:

```
$ pinctrl set 24 ip pu; pinctrl get 24
24: ip    -- | lo
```

Control test on an unused pin confirms pull configuration works on this board:

```
$ pinctrl set 22 ip pu; pinctrl get 22
22: ip    -- | hi
$ pinctrl set 22 ip pd; pinctrl get 22
22: ip    -- | lo
```

So GPIO22 follows the pulls; GPIO24 does not. BUSY is being held low by the hardware, not floating. (Note: the -- pull field is expected — BCM2835 pull registers are write-only and cannot be read back.)

Raspberry Pi power is clean throughout:

```
$ vcgencmd get_throttled
throttled=0x0
$ vcgencmd measure_volts core
volt=1.3500V      (varies 1.20-1.35V with load - normal DVFS)
```

No undervoltage detected at any point, on either power supply.

Reset timing (per Waveshare FAQ) — tested, no effect

The wiki FAQ entry “*Why is the BUSY pin always busy?*” suggests shortening the RST low period, noting that the power-off switch in the driver circuit can cause the board to lose power if reset is held low too long.

This was tested across six durations. After `module_init()` (PWR asserted), BUSY reads **ready** in every case:

```
RST low  2000us -> BUSY [1]
RST low  1000us -> BUSY [1]
RST low   500us -> BUSY [1]
RST low   200us -> BUSY [1]
RST low   100us -> BUSY [1]
RST low    50us -> BUSY [1]
```

Reset is therefore working correctly at any timing, and this FAQ item does not apply. The panel powers up, responds to reset, and reports ready. The failure is specifically that it never completes 0x04 (POWER_ON).

What has been ruled out

- Panel defect (2 units)
 - HAT defect (2 units)
 - Raspberry Pi model and SoC generation (Zero 2 W, Zero W, and 3 Model B — three SoC generations, identical failure)
 - OS and architecture (64-bit and 32-bit Raspberry Pi OS)
 - Driver software (5 independent implementations, including Waveshare’s own)
 - SPI configuration (enabled, both spidev nodes present)
 - Interface Config switch position (verified at 0 / 4-line SPI)
 - Cable routing (factory interposer assembly, and direct panel-to-HAT connection)
 - Cable seating and orientation (reseated and inspected multiple times)
 - Reset timing (six RST low durations from 2000µs down to 50µs, per the wiki FAQ — BUSY reports ready in all cases)
 - Power supply (two independent sources, no undervoltage events)
-

Questions for Waveshare

1. **What does a permanently-asserted BUSY after command 0x04 (POWER_ON) indicate?** This is the precise failure point. The panel acknowledges the command and then never signals ready.
2. **What voltages should be present at the HAT’s breakout header (VCC, PWR) during a successful power-on,** so they can be verified with a multimeter?
3. **What should the panel-side bias rails (VGH, VDH, VDD, VDHR, VCOM — labelled on the panel FPC) read during POWER_ON?**

4. **Is the 10.85" (G) validated on Raspberry Pi Zero-class boards, and what supply current does the HAT require?** The failure occurs at the highest-current step of the sequence. The 4-colour (G) panel presumably draws more than the black/white 10.85", which is documented working on a Pi Zero 1W by third-party users. Waveshare's own product photography shows this HAT on a full-size Raspberry Pi.

A Raspberry Pi 3 Model B was also tested and failed identically, but that test showed an undervoltage indication and is therefore **not** considered conclusive on the power question. Retest pending with an adequate supply.

5. **Is there a known erratum** for this panel/HAT combination, or a required init-sequence change not reflected in the published demo code?
-

Environment

- Raspberry Pi Zero W (BCM2835), Raspberry Pi OS 32-bit (Trixie), kernel 6.18.34+rpt-rpi-v6
- Previously: Raspberry Pi Zero 2 W, Raspberry Pi OS 64-bit
- WiringPi 3.18 (maintained fork) for C builds
- SPI enabled via `dtparam=spi=on`